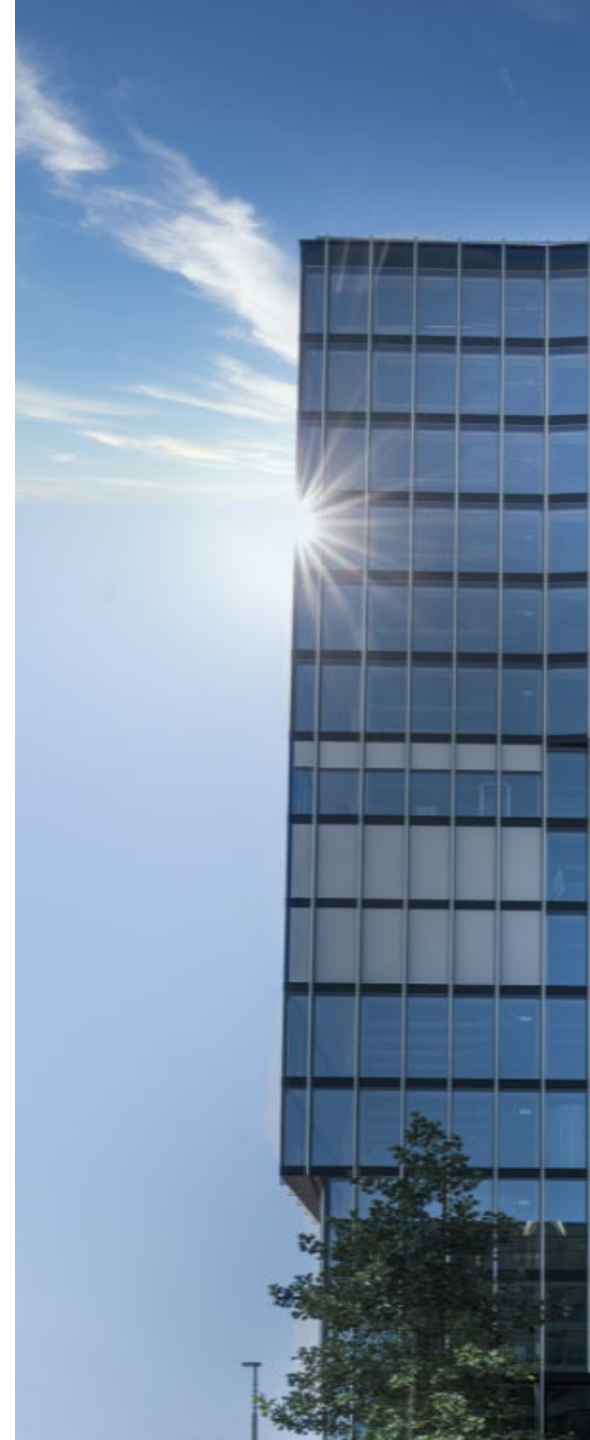


Objektorientierte Programmierung

Neuerungen bis Java 25 LTS

Was gibt es Neues seit Java 11 LTS

Roland Gisler



Inhalt

- Releasing und Weiterentwicklung von Java
- Seit Java 11 finalisierte und für OOP relevante JEPs:
 - Java 14: JEP 361 – **switch** und switch-Expressions
 - Java 15: JEP 378 – Text Blocks
 - Java 16: JEP 394 – Pattern Matching for **instanceof**
 - Java 16: JEP 395 – Records
 - Java 17: JEP 409 – Sealed Classes
 - Java 21: JEP 440 – Record Patterns
 - Java 21: JEP 441 – Pattern Matching for **switch**
 - Java 22: JEP 456 – Unnamed Variables & Patterns
 - Java 25: JEP 512 – Compact Source Files and Instance Main Methods
 - Java 25: JEP 513 – Flexible Constructor Bodies
- Weitere relevante Neuerungen / Zusammenfassung

Releasing von Java

Time-Based Releasing von Java (Oracle)

- Java wird seit September 2017 im Format: **\$FEATURE.\$INTERIM.\$UPDATE.\$PATCH** versioniert.
 - **FEATURE**: Inkrementeller Counter, aktuell bei **25** (2025)
 - **INTERIM**: Bleibt derzeit bei **0** (Sinn?).
 - **UPDATE**: Inkrementeller Counter für Bugfixes.
 - **PATCH**: Patch/Build-Nummer.
- Langfristiger Release-Plan:
 - Feature Releases **alle 6** Monate (jeweils März und September).
 - Diese sind nur ein halbes Jahr verfügbar.
 - Updates jeweils +1/+3 Monate (Apr./Jul. und Okt./Jan.).
 - Alle **zwei** Jahre wird der September-(Feature-)Release als ein **LTS** (Long Time Support) deklariert.
 - Nur für LTS gibt es über zwei (oder drei) Jahre Bugfixes.
- Nächster LTS wird die Version **29** im September **2027** sein.



Weiterentwicklung von Java

- Java wird über sogenannte «**JDK Enhancement-Proposal**» (JEP) weiterentwickelt. Dieser Prozess ist im [JEP 1](#) dokumentiert.
- Ein JEP kann sich (meistens) über mehrere Java-Versionen hinziehen, und hat dann «experimental»-Status.
 - «experimental»-Features müssen z.T. bewusst aktiviert werden.
- Was sich in diesem Status befindet, kann jederzeit auch wieder ändern (inkompatible Änderungen möglich).
- Abgeschlossene JEPs werden als «final» markiert, sind ab dann aktiviert (verfügbar), und somit nicht mehr veränderbar.
- Spannend: Auch LTS-Versionen können noch/wieder offene JEPs enthalten. Das ermöglicht langfristige Weiterentwicklung.

Die beste Quelle mit allen Details

- Die Original-Übersicht mit allen Sprachänderungen (seit Java 9) ist auf der folgenden Seite zu finden:

The screenshot shows a web browser displaying the Oracle Java Language Updates page for Java SE 25. The page title is "Java Language Updates" and the URL is <https://docs.oracle.com/en/java/javase/25/language/java-language-changes-release.html#GUID-6459681C-6881-45D8-B0DB-395D1BD6DB9B>. The page is divided into three main sections: a left sidebar with a table of contents, a main content area, and a right sidebar with a list of links.

Left Sidebar (Table of Contents):

- Title and Copyright Information
- Preface
- 1 Java Language Changes Summary
- 2 Java Language Changes by Release**
- 3 Preview Features
- 4 Module Import Declarations
- 5 Flexible Constructor Bodies
- 6 Compact Source Files and Instance main Methods
- 7 Sealed Classes
- ▶ 8 Pattern Matching
- 9 Record Classes
- 10 Unnamed Variables and Patterns
- 11 Switch Expressions and Statements
- 12 Text Blocks
- 13 Local Variable Type Inference
- 14 Safe Casting with instanceof and switch

Main Content Area:

2 Java Language Changes by Release

This section describes in detail the updated language features in Java SE 9 and subsequent releases.

Java Language Changes for Java SE 25

Feature	Description	JEP
When Clauses Type Patterns with Primitive Types Primitive Types in Record Patterns Primitive Values in switch Expressions and Statements Safe Casting with instanceof and switch	First previewed in Java SE 23 and previewed again in Java SE 24, this feature is re-previewed for this release without any significant changes.	JEP 507: Primitive Types in Patterns, instanceof, and switch (Third Preview)
Module Import Statements	First previewed in Java SE 23, and previewed again in Java SE 24, this feature is permanent in this release without any significant changes.	JEP 511: Module Import Declarations
Compact Source Files and Instance Main Methods	First previewed in Java SE 21 as <i>JEP 445: Unnamed</i>	JEP 512: Compact Source Files and Instance Main

Right Sidebar (On This Page):

On This Page

2 Java Language Changes by Release

- [Java Language Changes for Java SE 25](#)
- [Java Language Changes for Java SE 24](#)
- [Java Language Changes for Java SE 23](#)
- [Java Language Changes for Java SE 22](#)
- [Java Language Changes for Java SE 21](#)
- [Java Language Changes for Java SE 20](#)
- [Java Language Changes for Java SE 19](#)
- [Java Language Changes for Java SE 18](#)
- [Java Language Changes for Java SE 17](#)
- [Java Language Changes for Java SE 16](#)
- [Java Language Changes for Java SE 15](#)

<https://docs.oracle.com/en/java/javase/25/language/java-language-changes-release.html>

Java 14: JEP 361 – switch und switch-Expressions

switch und Switch-Expressions

- [JEP 361](#) wurde in Java 14 finalisiert.
- Das **switch**-Statement wurde funktional erweitert. Neu sind pro **case** auch mehrere Optionen möglich.
 - Diese Optionen werden durch Kommas separiert.
 - Labels (mit Doppelpunkt) mit «fall through» (**break** notwendig).
- Das **switch**-Statement kann neu auch als Expression (mit einem Rückgabewert) geschrieben werden.
 - Verkürzte Schreibweise mit Arrow-Labels (->), dann **kein** «fall through mehr! **yield** zur direkten Rückgabe aus Codeblöcken.
 - **switch**-Expressions müssen **vollständig** sein, d.h. alle möglichen Fälle/Werte müssen geprüft werden!
- Das sind alles vorbereitende Änderung für «pattern matching».

Beispiel 1: switch-Statement - Mehrfachselektion

- Liefert zu einer Tagnummer (`int value`) die Art des Tages:

```
switch (value) {  
    case 1, 2, 3, 4, 5:           // Neu mit multi-case  
        typ = "Arbeitstag";  
        break;                  // break weiterhin notwendig  
    case 6, 7:  
        typ = "Wochenende";  
        break;  
    default:  
        typ = "Unerlaubte Tagnummer";  
}
```

- Das `default` ist in diesem Fall **optional**, da nicht alle `int`-Werte berücksichtigt werden müssen.
- Das `break` ist aber weiterhin **notwendig**, da wir ansonsten «fall through» hätten.

Beispiel 2: switch-Expression – mit Arrow Labels

- Stark verkürzte Syntax mit Arrow-Labels (nun **ohne** «fall through»!):

```
String daytype = switch (value) {  
    case 1, 2, 3, 4, 5 -> "Arbeitstag";  
    case 6, 7          -> "Wochenende";  
    default            -> "Unerlaubte Tagnummer";  
}
```

- Expression: **switch** liefert den Wert zurück (hier z.B. **String**).
- Das **default** ist in diesem Fall **zwingend**, da alle möglichen **int**-Werte berücksichtigt werden müssen.
- Ein **break** ist hier nicht mehr notwendig, da wir **kein** «fall through» mehr haben!

Beispiel 3: switch-Expression – mit Codeblock und yield

- Pro **case** (oder **default**) sind auch Codeblöcke {...} möglich:

```
String daytype = switch (value) {  
    case 1, 2, 3, 4, 5 -> {  
        String name = "Arbeitstag(" + value + ")";  
        LOG.info("Es ist {}", name);  
        yield name;  
    }  
    case 6, 7 -> "Wochenende";  
    default -> "Unerlaubte Tagnummer";  
}
```

- Bei Verwendung eines (optionalen) Codeblockes muss das Schlüsselwort **yield** genutzt werden, um die **switch**-Expression mit dem gewünschten Wert zu verlassen.
 - Analog zu einem **return** aus einer ganzen Funktion.

Java 15: JEP 378 - Text Blocks

Text Blocks

- JEP 378 wurde in Java 15 finalisiert.
- Mit Text Blocks ist es möglich, **vorformatierte** Textblöcke direkt im Quellcode abzulegen (mehrzeilig, eingerückt, Sonderzeichen).
- Das erspart das mühsame Konkatenieren von mehreren Zeilen zu einem gesamten Block, und das «escapen» von Sonderzeichen.
- Somit können z.B. SQL-Statements oder HTML-Fragmente etc. sehr übersichtlich und gut lesbar formatiert direkt im Code eingefügt werden.
- Fun fact: Selten wurde über einen vermeintlich einfachen JEP so lange gestritten!
 - Herausforderung: Die syntaktischen Regeln für die «Grundeinrückung».

Text Blocks – Beispiel: Klassisch vs. Textblock

- Klassische Definition eines SQL-Statement in einen **String**

```
String result = "select \"name\", \"vorname\"\\n"  
                + "    from \"person\" p\\n"  
                + "    where (p.\"name\" like 'M%')\\n"  
                + "    order by p.\"name\";";
```

- Einrückungen unübersichtlich und fehleranfällig.
- Escaping (Anführungszeichen, Zeilenumbruch) schwer leserlich.

- Neue Syntax mit Textblock (**drei** Anführungszeichen):

```
String result = """  
    select "name", "vorname"  
    from "person" p  
    where (p."name" like 'M%')  
    order by p."name";  
""";
```

- Deutlich übersichtlicher und lesbarer, oder?

Java 16: JEP 394 - Pattern Matching for instanceof

Pattern Matching for instanceof

- [JEP 394](#) wurde in Java 16 finalisiert.
- Hinter dem durch **instanceof** zu prüfenden Typ, kann direkt der Name einer Variable deklariert werden.
- Passt der Typ des geprüften Objektes, erhält die Variable die auf den entsprechenden Typ gecastete Objektreferenz.
- Scope dieser Variablen wird speziell gesteuert (flow scoping):
 - Je nach Expression und weiterem Verlauf bleibt der Scope der Variable sinnvoll erhalten.
- Effekt: Die nach **instanceof** häufigen, expliziten (Down-)Casts können entfallen. Der Code wird deutlich kürzer und lesbarer.
 - Wirkt sich z.B. auch in der Vereinfachung von **equals()**-Methoden aus!

Pattern Matching – Beispiel 1

- Alter, bisheriger Code (ohne Pattern Matching, bis Java 15):

```
final Object object = new Point(2,4);
if (object instanceof Point) {
    final Point point = (Point) object;           // expliziter cast
    System.out.println(point + " ist ein Punkt!");
}
```

- Neue Syntax (mit Vereinfachung, seit Java 16):

```
final Object object = new Point(2,4);
if (object instanceof Point point) {
    // point steht hier bereits gecastet bereit!
    System.out.println(point + " ist ein Punkt!");
}
// Scope von point aufgelöst
```

- Unscheinbare, aber **sehr schöne** Erweiterung!

Beispiel 2: Vereinfachte equals()-Methoden

- **instanceof** kann in einer Expression direkt mit weiteren logischen Verknüpfungen (AND, OR) kombiniert werden.
 - Erinnerung: Es werden nur erreichbare Ausdrücke ausgewertet!
- Beispiel: **equals()**-Methode von einem **Point(int, int)**:

```
@Override
public boolean equals(final Object object) {
    if (object == this) {
        return true;
    }
    return (object instanceof Point point)
        && (point.x == this.x)
        && (point.y == this.y);
}
```

Java 16: JEP 395 - Records

Records

- [JEP 395](#) wurde in Java 16 finalisiert.
- Idee: Einfache und unveränderliche (immutable) Datenklassen (vgl. DTO/VO* etc.) sollen mit sehr wenig Code deklarativ erstellbar sein.
 - Alle Attribute sind **privat**, Zugriffsmethoden werden erzeugt.
 - Mit **equals()** und **hashCode()**-Methoden (Wertegleichheit).
 - Inklusive **toString()**.
- Können bei Bedarf mit zusätzlichen Methoden ergänzt werden.
- Kontroverser Entscheid: Die Zugriffsmethoden halten sich nicht an die Getter-Namensgebung, sondern tragen nur den Namen des Attributes.
 - Einsatz von Records z.B. bei Serialisierung (JSON/XML etc.) oder JPA derzeit noch erschwert (diese erwarten normale Getter).

Beispiel für eine record-Klasse

- Neues Schlüsselwort **record** (statt **class**, wie bei **enum**).
- Definition einer **Person** mit Attributen für **name** und **vorname**:

```
public record Person(String name, String vorname) {  
    // Mehr braucht es nicht!  
}
```

- Die Klasse kann dann wie folgt genutzt werden:

```
Person p = new Person("Muster", "Hans");  
String name = p.name() + " " + p.vorname();  
boolean b = p.equals(new Person("Lutz", "Pia"));  
int hash = p.hashCode();  
String output = p.toString();
```

Java 17: JEP 409 - Sealed classes

Sealed classes

- [JEP 409](#) wurde in Java 17 finalisiert.
- Eine Superklasse (oder auch Interface) kann die erlaubten Spezialisierungen (bzw. Implementierungen) explizit definieren.
- Sinnvoll bei «geschlossenen» Typmengen (vgl. Enumerationen), bei welchen eine (ggf. abstrakte) Superklasse als Typ dennoch breit verfügbar sein soll.
- Sealed Classes stellen **keine** Konkurrenz zu **final** dar.
 - Soll eine Klasse nicht spezialisierbar sein, **finalisiert** man sie.
- Interessantes Detail: Eine Spezialisierung einer sealed class kann die Vererbung auch wieder öffnen! Schlüsselwort **non-sealed**
- Sealed classes sind primär eine Vorbereitung für zukünftige Erweiterungen im Bereich des «pattern matching».

Sealed classes – Beispiel Periodensystem

- Die Basisklasse **Element** ist ein sinnvoll einzusetzender Typ. Die davon spezialisierten konkreten Elemente sollen aber eine geschlossene Menge darstellen.
 - Es soll **nicht** möglich sein, ein neues Element zu «erfinden».

```
public abstract sealed class Element
    permits Wasserstoff, Helium, Lithium, Beryllium,...    // etc.
{
    // Implementation
}
```

- **Element** ist hier sinnvollerweise **abstract**, muss aber nicht sein.
- Die konkreten Spezialisierungen (im selben Package) wären in diesem Beispiel dann sinnvollerweise **finalisiert** (**final**).

Sealed classes – Weitere Beispiele / Varianten

- Die Spezialisierungen können auch explizit in andere Packages erzwungen werden (aber immer gleiches Modul):

```
package ch.hslu.oop.basis;  
  
public abstract sealed class Element  
    permits ch.hslu.oop.elemente.Wasserstoff,  
            ch.hslu.oop.elemente.Helium,... { ... }
```

- Oder (wenn es wenige **und** kleine Klassen wären) auch direkt **innerhalb** der Klasse definiert werden (**permit** entfällt dann):

```
public abstract sealed class Element {  
    final class Wasserstoff extends Element { ... }  
    final class Helium extends Element { ... }  
    ...  
}
```

Wichtige Hinweise zu sealed classes

- Der JEP 409 ist relativ umfangreich und bietet weitreichende Möglichkeiten.
Darum: Zurückhaltend und nur wohlüberlegt nutzen!
 - Nicht alles, was geht, ist auch sinnvoll!
- Zum Beispiel kann man mit einem sealed Interface auch festlegen, welche Klassen das Interface implementieren dürfen!
 - Ein «Mix» zwischen Klassen und Interfaces somit auch möglich.
- Spezialisierungen einer sealed class könnten in einem anderen Package liegen, müssen aber im selben Modul (JPMS) sein!
- Erinnerung: Sealed classes sind primär eine Vorbereitung zu «pattern matching».

Java 22: JEP 456 – Unnamed Variables & Patterns

Unnamed Variables & Patterns

- [JEP 456](#) – wurde in Java 22 finalisiert.
- Man kann neu “unbenannte” Variablen mit einem “_” (Tiefstrich) als Platzhalten nutzen.
- Der Scope dieser unbenannten Variablen ist an Ort und Stelle, das Symbol “_” kann somit mehrfach ohne Konflikte genutzt werden.
- Auf die unbenannte Variable kann nirgends zugegriffen werden.
- Dieses Feature wurde (wie vieles) hauptsächlich für das noch in der (Weiter-) Entwicklung befindliche «pattern matching» ergänzt.
- Es lässt sich aber universell und überall nutzen, speziell dann wenn aus syntaktischen Gründen an Stellen ein Variablenname deklariert werden muss, die Variable im weiteren Verlauf des Codes aber nicht verwendet wird.
 - Zum Beispiel weil es nur ein Nebeneffekt ist, bzw. nur Nebeneffekte erfolgen.

Beispiele für «Unnamed Variablen»

- Ein weiteres Element wird in ein bestehendes Set eingefügt. Ob das Einfügen geklappt hat oder nicht, ist für den weiteren Verlauf aber nicht relevant:
 - Das Resultat der `add()`-Methode wird «bewusst» konsumiert (bei Java optional).

```
Set<String> uniqueList = Set.of("Alpha", "Beta", "Gamma");  
boolean _ = uniqueList.add("Omega");
```

- Man iteriert über die Liste, um als Seiteneffekt die Anzahl Elemente zu zählen. Da man dafür die einzelnen Elemente nicht benötigt, wird eine unbenannte Variable deklariert:

```
int count = 0;  
for (String _ : uniqueList) {  
    count++;  
}  
IO.println("Anzahl: " + count);
```

- Man kann so bewusste Nebeneffekte besser sichtbar machen.

JEP 512: Compact Source Files and Instance Main Methods

Compact Source Files and Instance Main Methods

- [JEP 512](#) – wurde in Java 25 finalisiert.
- Ziel: Java für kleine, einfache (Script-)Programme brauchbar machen, und die Einstiegshürden minimieren.
 - Über Aufwand und Nutzen kann man streiten! Java ist nun mal **keine** Scriptsprache.
- Bisher sah ein «HelloWorld» in Java (in einem Package) wie folgt aus:

```
package ch.hslu.oop.oop01;

public final class HelloWorld {

    public static void main(final String[] args) {
        System.out.println("Haben Sie Fragen?");
    }
}
```

Vereinfachungen für kleine, «Ein-Klassen-Programme»

- Die Datei darf **keine** Package-Deklaration enthalten (somit im **Default**-Package).
- Quellcode in einer Java-Datei (z.B. «DemoJava25.java»), egal wo abgelegt.

```
void main() {  
    IO.println("Haben Sie Fragen?");  
}
```

- Ergebnis der Vereinfachungen:
 - Klassendefinition ist **nicht** mehr **nötig**.
 - **main()**-Methode neu **nicht** mehr statisch und hat auch keine Argumente mehr.
 - Eine **IO**-Klasse neu in **java.lang**-Package (auto-import), mit Wrapper-Methoden.
- Kann mit **java DemoJava25.java** gestartet werden.
 - Die Klasse wird beim Start automatisch **in-memory** kompiliert.
 - Es wird **keine** persistente class-Datei erzeugt.

JEP 513: Flexible Constructor Bodies

Flexible Constructor Bodies

- [JEP 513](#) – wurde in Java 25 finalisiert.
- Bisher war die Reihenfolge von Konstruktoren **bewusst** sehr restriktiv («top-down»): Als erstes Statement musste **immer** der **super(...)**-Aufruf erfolgen.
- Ein Nachteil davon ist, dass somit vorgängige Parameterprüfungen (fail-fast) nicht möglich waren, und somit unnötige Arbeit geleistet wurde.
 - Trotzdem war auch damit «prenatale Kommunikation» und Inkonsistenzen möglich.
 - Erinnerung: Keine überschreibbaren Methoden in Konstruktoren aufrufen!
- Mit dem JEP 513 hat man die Regeln aufgeweicht, und die Konstruktoren logisch in einen «Prolog», «Body» und «Epilog» aufgeteilt. Die Aufrufreihenfolge ändert zu «bottom-up».
 - Inkonsistenzen sind noch immer möglich, gleichzeitig können wir aber effizientere Lösungen implementieren.

Beispiel für neue Möglichkeiten in Konstruktoren

- Voraussetzung: Eine einfache **Person**-Klasse (Name und Vorname), spezialisiert zu **Mitarbeiter** mit zusätzlichen Attributen (z.B. Alter).
- Konstruktor von **Mitarbeiter**:

```
public Mitarbeiter(String vorname, String nachname, int alter) {  
    // Prolog (war vorher nicht möglich!)  
    if (alter < 18) {  
        throw new IllegalArgumentException("Zu jung!");  
    }  
    // Body  
    super(vorname, nachname);  
    // Epilog  
    this.alter = 0;  
}
```

Weitere relevante Neuerungen

Erweitertes, leichtgewichtiges Treading-Modell

- Mit Java 21 wurde die API für Multithreading massiv erweitert.
- Neu kann man entscheiden, ob man
 - «schwergewichtige», native OS-Threads will (default), *oder*
 - leichtgewichtige Java-Threads (nur in der JVM, vgl. fibers)
- Mit Java 25 wurden «scoped blockes» für kurzfristige, abgesicherte Nebenläufigkeit eingeführt (keine Sprachänderung, sondern API-Erweiterungen).
- Da Nebenläufigkeit / Multi-Threading in OOP noch kein Thema ist, gehen wir hier aber nicht näher darauf ein. Siehe stattdessen:
 - Modul AD (Algorithmen und Datenstrukturen), oder
 - Modul ADS (Algorithmen, Datenstrukturen und Security)

Zusammenfassung

- Das **switch**-Statement wird wieder interessanter.
- Die switch-Expressions (mit «pattern matching») können Code deutlich vereinfachen.
- Text Blocks sind nett, wenn man sie mal braucht.
- Records ersparen uns für simple (!) Werteklassen einiges an Programmieraufwand.
- Sealed classes und «unnamed variables» werden mit noch folgenden «pattern matching»-Erweiterungen eine grosse Rolle spielen, aber vermutlich nicht so gross in der «klassischen» OO-Programmierung?
- Flexiblere Konstruktor-Bodies vereinfachen die Programmierung
- Viele weitere JEPs sind derzeit noch im Preview und in Entwicklung!
- Entwicklungen zur (vereinfachten, effizienteren) Nebenläufigkeit hier nicht vertieft!



Fragen?